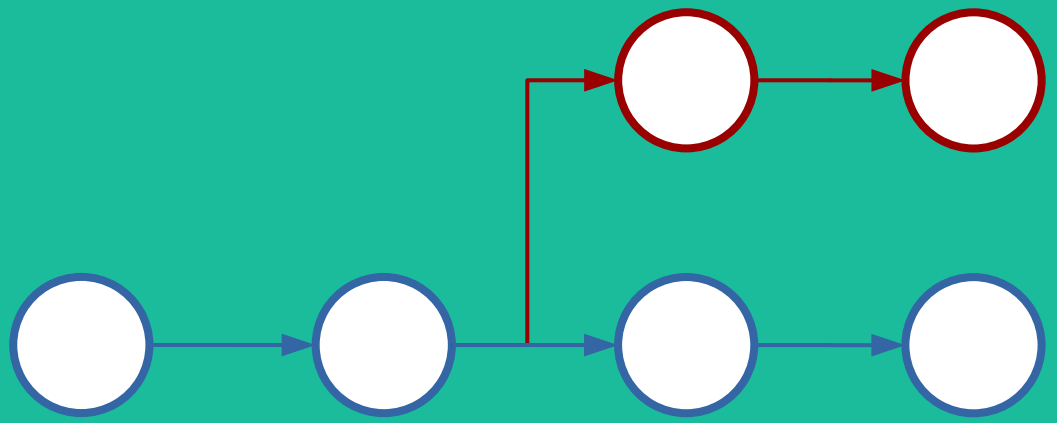


# 3 – Embranchements

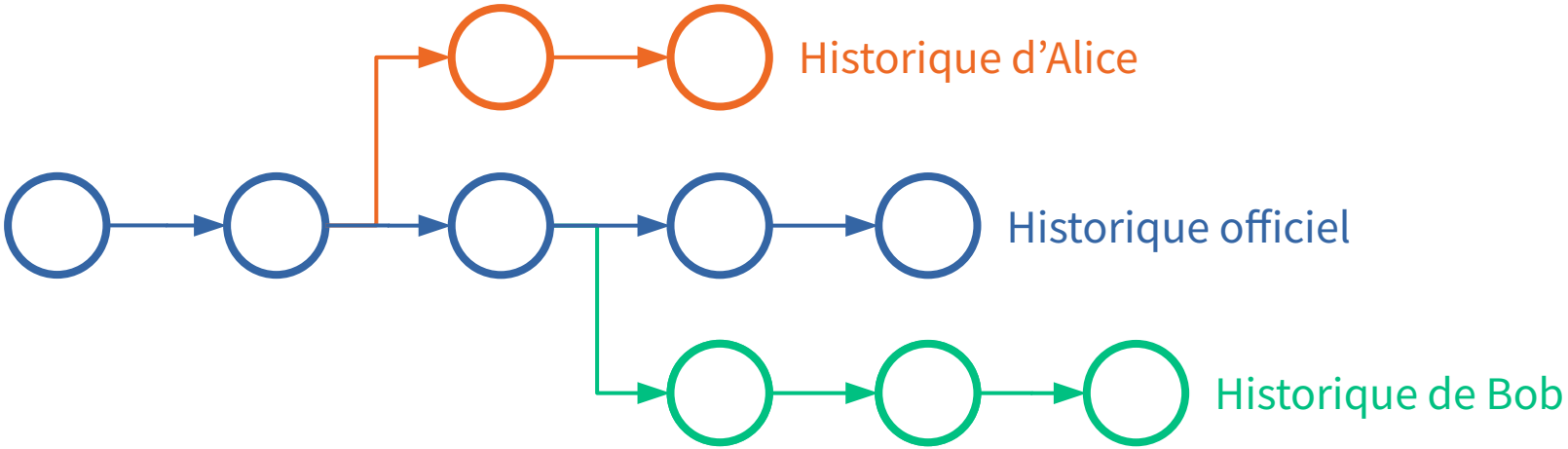
Hadrien Grasland

IJCLab – Orsay

23/11/2023

# Une divergence naturelle

- **Tout historique en réseau finit par diverger :**




- **Différentes stratégies possibles :**
  - Centralisé : On essaie de le cacher à l'utilisateur
  - Distribué : On fournit des outils pour le gérer

# Notion de branche

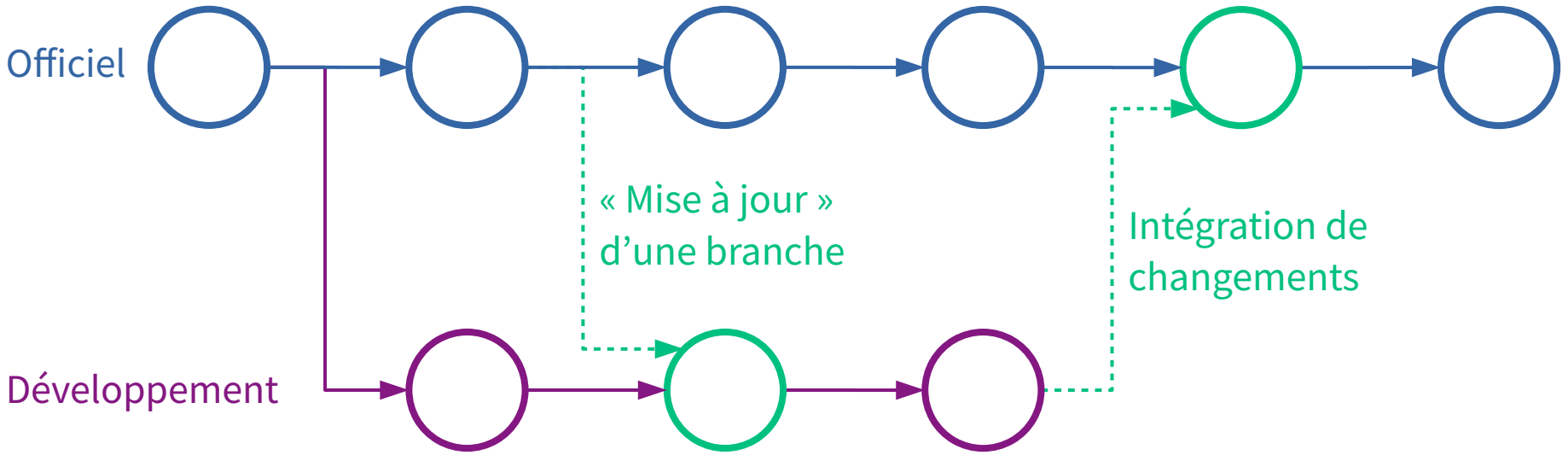
- **Branche = Chaîne de commits**
  - Peut partir de rien, comme *master*…
  - …ou, plus souvent, d'un commit d'une autre branche
  - On peut ajouter des commits en bout de chaîne
- **Git désigne les branches par leur extrêmité**
  - Suffisant car chaque commit connaît ses parents
  - Plus économe que d'annoter chaque commit

# Créer des branches

- **Pourquoi ?**
  - Garder de côté un ou plusieurs historique(s) officiel(s)
  - Mener des développements en parallèle de ceux-ci
- **Comment ?**
  - Vaste question des méthodes de travail (*workflows*)
  - Idée récurrente : Créer une branche pour un petit projet
  - Ex : Chapitre d'un livre, résolution d'un problème…

# Fusion de branches

- **Après avoir divergé, il faut pouvoir converger :**



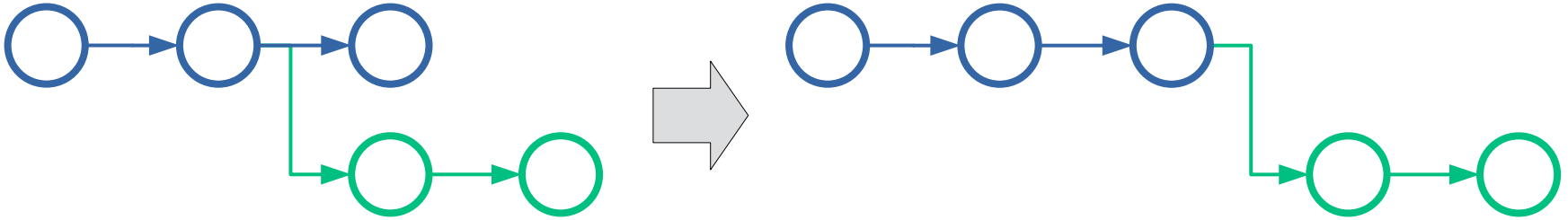

- **Méthode la plus simple : La fusion (*merge*)**
  - Un commit intègre les nouveautés d'une autre branche

# Fusion automatique et conflits

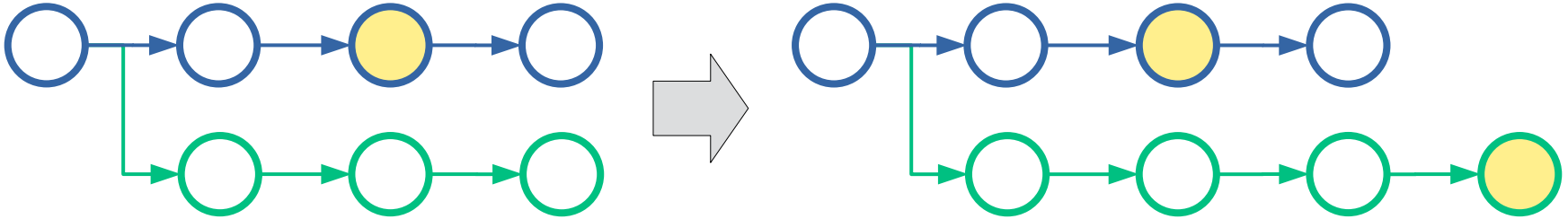
- **Une fusion remonte à un ancêtre commun puis…**
  - Examine les modifications apportées à chaque fichier
  - Juge si ces modifications semblent compatibles :
    - Si oui, crée un commit qui combine tous les changements
    - Si non, il y a un conflit que l'utilisateur doit résoudre
- **Le texte brut joue un rôle clé dans ce processus :**
  - Modification = Bloc de lignes contigües (« hunk »)
  - Compatible = Modifications sur des blocs éloignés

# Au-delà des fusions

- **D’autres méthodes de synchronisation existent :**
  - *Rebase* : Reconstruire une branche sur un autre ancêtre



  - *Cherry-pick* : Appliquer sélectivement des commits



# Démo : Les branches avec gitg et meld

# TP : Manipulation de branches